Machine Intelligence Research
www.mi-research.net

21(3), June 2024, 549-570
DOI: 10.1007/s11633-023-1426-8

An Empirical Study on Google Research Football Multi-agent Scenarios

Yan Song[1*] He Jiang[2*] Zheng Tian[3] Haifeng Zhang[1] Yingping Zhang[4]
Jiangcheng Zhu[4] Zonghong Dai[4] Weinan Zhang[5] Jun Wang[6]

[1] Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China

[2] Digital Brain Lab, Shanghai 200001, China

[3] ShanghaiTech University, Shanghai 200001, China

[4] Huawei Cloud, Guiyang 550003, China

[5] Shanghai Jiao Tong University, Shanghai 200001, China

[6] University College London, London WC1E 6PT, UK

Abstract: Few multi-agent reinforcement learning (MARL) researches on Google research football (GRF)[1] focus on the 11-vs-11 multi-agent full-game scenario and to the best of our knowledge, no open benchmark on this scenario has been released to the public. In this work, we fill the gap by providing a population-based MARL training pipeline and hyperparameter settings on multi-agent football scenario that outperforms the bot with difficulty 1.0 from scratch within 2 million steps. Our experiments serve as a reference for the expected performance of independent proximal policy optimization (IPPO)[2], a state-of-the-art multi-agent reinforcement learning algorithm where each agent tries to maximize its own policy independently across various training configurations. Meanwhile, we release our training framework Light-MALib which extends the MALib[3] codebase by distributed and asynchronous implementation with additional analytical tools for football games. Finally, we provide guidance for building strong football AI with population-based training[4] and release diverse pretrained policies for benchmarking. The goal is to provide the community with a head start for whoever experiment their works on GRF and a simple-to-use population-based training framework for further improving their agents through self-play. The implementation is available at https://github.com/Shanghai-Digital-Brain-Laboratory/DB-Football.

Keywords: Multi-agent reinforcement learning (RL), distributed RL system, population-based training, reward shaping, game theory.

Citation: Y. Song, H. Jiang, Z. Tian, H. Zhang, Y. Zhang, J. Zhu, Z. Dai, W. Zhang, J. Wang. An empirical study on google research football multi-agent scenarios. *Machine Intelligence Research*, vol.21, no.3, pp.549–570, 2024. http://doi.org/10.1007/s11633-023-1426-8

1 Introduction

Football is one of the most popular and long-lasting sports worldwide. In a football game, two teams of 11 players compete for victory by scoring more goals. This often requires high-level cooperation in both offense and defense. With its complex nature in agent control, competition and cooperation, football has also been regarded as one of the most challenging testbeds for multi-agent intelligence research. Our research is conducted on the football game engine proposed by Kurach et al.[1] named the Google research football (GRF), which is the only realistic simulator that is available for public research. It bears great similarity to well-known football games such as FIFA and Real Football, vividly modelling the dynamics of ball and player as well as their interaction such as passing, shooting, etc. Many previous studies have examined their algorithms on simple academy scenarios provided by the GRF engine, which only involves a small number of players and is restricted to a single offensive and defensive round. However, the more challenging multi-agent full-game scenario where two teams of 11 players compete against each other in 90 minutes[1], is far less explored due to the following major difficulties from the perspective of reinforcement learning:

1) The sparsity of rewards;
2) The long duration of game playing;
3) The high stochasticity in the game state transition.

From the perspective of multi-agent system:

4) Role or credit assignment to players;

Research Article
Manuscript received on November 25, 2022; accepted on February 10, 2023; published online on January 12, 2024
Recommended by Associate Editor Maoguo Gong
Colored figures are available in the online version at https://link.springer.com/journal/11633

* These authors contribute equally to this work

© Institute of Automation, Chinese Academy of Sciences and Springer-Verlag GmbH Germany, part of Springer Nature 2024

[1] 90 minutes game clock corresponds to 3 000 discretized steps

5) The exponential increase in policy search space resulted from multi-agent cooperation and competition.

To our knowledge, our work is the first to scale a multi-agent reinforcement learning algorithm to the full-game GRF scenario and successfully beat the strongest built-in AI with 1.0 difficulty by training from scratch. In particular, we contribute an empirical comparison of various training configurations against built-in AI in the full-game scenario and provide technical advice on how to build up even stronger AI through self-play by our population-based training framework.

Our goal is to provide the community with a head start for whoever attempts to experiment with their works on GRF and a simple-to-use population-based training framework to further improve their AI through self-play. We also believe our research will provide great insights into the study of scalability issues in multi-agent learning.

To summarize, we listed our main contribution as follows:

1) We demonstrate the scalability and effectiveness of independent PPO by beating the built-in AI with 1.0 difficulty in the 11-vs-11 full-game scenario and conduct thorough empirical studies on various training settings.

2) We further improve our agents by population-based self-play[4] in the GRF 11-vs-11 scenario and obtain stronger policies of different playing styles that can serve as good initializations or baselines for future research.

3) We release our distributed population-based reinforcement learning framework Light-MALib, with diverse pretrained models and data analytical tools specifically targeting on GRF research.

2 Related works on GRF

2.1 Reinforcement learning

Techniques for reinforcement learning (RL) have matured rapidly over the last few years, yielding high-level performance in various tasks such as games[4–7], robotics[8], advertising[9], mobile networks[10], etc. Reinforcement learning methods search for optimal policies by maximizing the expected cumulated rewards collected by the agent while interacting with the external environment[11]. This can be done by learning to estimate the goodness of state-action pairs[11, 12] and directly optimizing the policy function[13]. Regarding the number of controllable agents, reinforcement learning can be categorized into two classes, single-agent RL (SARL) with one controllable agent and multi-agent RL (MARL)[2, 7, 14] where multiple agents generate actions based on their observations either individually or cooperatively. Google research football includes both SARL and MARL scenarios and has been adopted as a testbed for AI competitions as well as academic research.

2.2 Online competition on GRF

Single-agent RL (SARL) Google research football 2020 Kaggle competition[15] was held by the famous Manchester City F.C., UK, which attracted over 1 000 team participation in the single-agent 11-vs-11 full-game scenario. In this task, one side of the team needs to control merely the designated player (one at each step) and all of its teammates are controlled by the built-in AI. Among all the participants, reinforcement learning methods have been widely used and the winner – WeKick[16] adopted the self-play reinforcement learning approach. This competition has largely boosted GRF research from an engineering aspect and some of the open-sourced SARL submissions have provided us with valuable suggestions on both feature engineering and action masking at an early stage. Recently, MARL CoG 2022 GRF tracks[17] were held at the Conference of Games on multi-agent RL settings (5-vs-5 and 11-vs-11 full-game scenarios) with an increase in the number of controllable players. Unfortunately, few teams share their implementations. This encourages us to open-source our works and provide guidance on multi-agent GRF tasks.

2.3 Academic research on GRF

As a large-scale multi-agent RL testbed involving both competitive and cooperation tasks, GRF has been experimented with in many academic studies as listed in Table 1. Many of them test on relatively simpler academy scenarios. Among works in 11-vs-11 multi-agent scenarios, Niu et al.[26] measure the average reward of their method but do not solve the game; Huang et al.[27] apply imitation learning on WeKick single-agent demonstrations and achieves a high win rate in the 11-vs-11 multi-agent full-game scenario. However, their performances are largely based on prior experiences, such as the expert demonstration and the action_set_v2 action space. The action_set_v2 is an official 20-dimensional action space that includes an extra built-in AI action choice compared to the 19-dimensional default action set. This action choice is likely to select valuable action according to built-in logic and therefore often results in good-quality exploration. One of our experiments also showed that action_set_v2 can indeed largely accelerate convergence at the early stage of training. Meanwhile, the baseline MAPPO algorithm they compare their methods to has terrible performance and this contradicts our experience.

In contrast, our work focuses on solving multi-agent football matches from scratch without any previous pretrained models and sticking to the default action set with each action choice corresponding to a unique move in a football game. To the best of our knowledge, we propose the first open benchmark for GRF 11-vs-11 multi-agent

Table 1 Academic research on GRF and ours

Paper	Research topics	Testing scenarios	Open-sourced
Li et al.[18]	Diversity	Academy 3-vs-1 with keeper, Academy counterattack hard,3-vs-1 with keeper (full field)	Yes
Niu et al.[19]	Communication	3-vs-2 scenario	Yes
Roy et al.[20]	Coordination	3-vs-2 scenario	No
Yang and Tang[21]	Reward shaping	Run_to_score_with_keeper, Pass-and_shoot_with_keeper, Run_pass-and_shoot_with_keeper,	No
Ruan et al.[22]	Coordination	3-vs-2, 3-vs-6, 5-vs-5 multi-agent scenarios	Yes
Wang et al.[23]	Reward shaping	5-vs-5 half-court offense	No
Jiang et al.[24]	Exploration	Academy 3-vs-1 with keeper, Academy 4-vs-2 with keeper, Academy counterattack hard	No
Pu et al.[25]	Cooperation	3-vs-2 scenario	No
Niu et al.[26]	Coordination	11-vs-11 full-game	No
Huang et al.[27]	Imitation	11-vs-11 full-game	Only pre-trained models and testing scripts
Ours	Benchmarking from scratch	11-vs-11 full-game	Training framework, trained model and testing scripts

full-game scenarios from scratch and the first public detailed implementation of agent training along with it. We also carry out population-based training to improve the general abilities of our policies and release our trained models.

In Section 3, we introduce our implemented algorithm. Section 4 contains details of our distributed framework. Section 5 contains the experimental results of MARL and Section 6 discusses population training which further improves the abilities of football AI.

3 Algorithm

3.1 Notations

Appendix A.1 lists the notations used in this paper.

3.2 Problem formulation

The multi-agent football scenario can be formulated as a decentralized partially-observed Markov decision process (DEC-MDP) problem. Similar to the definition of a Markov decision process, a DEC-MDP is defined by a 7-tuple $(n, \mathcal{S}, \mathcal{A}, \mathcal{O}, R, P, \gamma)$, where:

1) n is the total number of agents;

2) $\mathcal{S}$ is the state space and $s_t \in \mathcal{S}$ represents the state at timestep t;

3) $\mathcal{O}$ is the joint observation space of all agents and $o_t^i \in \mathcal{O}^i$ represents the observation of agent i at timestep t, where $\mathcal{O}^i$ is the observation space of agent i;

4) $\mathcal{A}$ is the joint action space of all agents, $a_t^i \in \mathcal{A}_t^i$ is the action taken by agent i at timestep t, where $\mathcal{A}^i$ is the action space of agent i;

5) $P\left(s_t|s_{t-1}, A_{t-1}\right)$ is the state transition probability that models the probability of state s_t after agents take joint action $A_{t-1} = \left(a_{t-1}^1, \cdots, a_{t-1}^n\right)$ at the previous state s_{t-1};

6) R is a set of reward functions and $r_t^i = R^i\left(s_t, A_t\right)$ is the reward received by agent i after agents take joint action $A_t = \left(a_t^1, \cdots, a_t^n\right)$ at state s_t;

7) γ is the discount factor.

3.3 Backbone MARL algorithm: Independent proximal policy optimization (IPPO)

Proximal policy optimization (PPO) has been widely used in both single-agent tasks[13] and multi-agent tasks[2]. In this work, we use PPO with parameter-sharing, where we learn a policy $\boldsymbol{\pi}_\theta$ and a value function V_ϕ (parametrized by θ and ϕ separately) shared by all agents. Specifically, $\boldsymbol{\pi}_\theta\left(a_t^i|o_t^i\right)$ denotes the probability of action a_t^i taken by agent i after observing o_t^i and $V_\phi\left(s_t\right)$ denotes the estimated return of state s_t. In the IPPO setting, each agent tries to maximize its own cumulative reward and the overall objective is

$$\theta \leftarrow argmax_\theta J\left(\theta\right) = \mathbf{E}_{a_t, s_t}\left[\sum_t \gamma^t R\left(s_t, a_t\right)\right]. \tag{1}$$

According to the loss derivation of PPO[13], the policy loss of IPPO can be computed as

$$L\left(\theta\right) = \sum_{i=1}^{n} \mathbf{E}_{s\sim\rho_{\theta_{old}}, a\sim\boldsymbol{\pi}_{\theta_{old}}}\left[\min\left(\frac{\boldsymbol{\pi}_\theta\left(a^i|s\right)}{\boldsymbol{\pi}_{\theta_{old}}\left(a^i|s\right)}\widehat{A}_t\ ,\right.\right.$$
$$\left.\left. clip\left(\frac{\boldsymbol{\pi}_\theta\left(a^i|s\right)}{\boldsymbol{\pi}_{\theta_{old}}\left(a^i|s\right)},\ 1-\varepsilon,\ 1+\varepsilon\right)\widehat{A}_t\right)\right] \tag{2}$$

where the clip operator ensures that the ratio $\frac{\pi_\theta\left(a^i|s\right)}{\pi_{\theta_{old}}\left(a^i|s\right)}$ is

within the range of $[1-\varepsilon, 1+\varepsilon]$ to avoid large parameter updates. Compared to (1), the IPPO policy loss $\mathcal{L}(\theta)$ adds tricks of 1) importance sampling $\frac{\pi_\theta(a^i|s)}{\pi_{\theta_{old}}(a^i|s)}$ to correct off-policy searching[11]; 2) clipping operation $clip(\cdot)$ to enable parameter regularization and 3) generalized advantage estimation (GAE)[28] $\widehat{A}_t$ to achieve bias-variance trade-off. The advantage estimation $\widehat{A}_t$ is defined as follows:

$$\begin{aligned}&\widehat{A}_t=\sum_{l=0}^{h}(\gamma\lambda)^l\delta_{t+l}\\&\delta_t=r_t(s_t,a_t)+\gamma V_\phi(s_{t+1})-V_\phi(s_t)\end{aligned}\tag{3}$$

where δ_t is the TD error at time step t and we set h such that $t+l$ reaches the end of the episode. Intuitively, both hyperparameters λ and γ determine bias-variance tradeoff and γ also determines the length of horizon. Detailed explanations of GAE can be found in [28].

The value loss function of IPPO is formulated as a clipped version of the mean square error (MSE) loss between the value function $V_\phi(\cdot)$ and the target state value $\widehat{V}_t$:

$$\begin{aligned}&\mathcal{L}^i(\phi)=\mathbf{E}_{s\ \rho_{\theta_{old}}}\left\{\min\left[\left(V_\phi(s_t)-\widehat{V}_t\right)^2,\right.\right.\\&\left.\left.\left(V_{\phi_{old}}(s_t)+clip\left(V_\phi(s_t)-V_{\phi_{old}}(s_t),-\varepsilon,+\varepsilon\right)-\widehat{V}_t\right)^2\right]\right\}\end{aligned}\tag{4}$$

where $\widehat{V}_t=\widehat{A}_t+V_\phi(s_t)$ is the sum of the advantage estimate and the current state value, which we call the GAE return. By minimizing $\mathcal{L}^i(\phi)$, the value function V_ϕ approaches $\widehat{V}_t$.

Additionally, an entropy regularization term $\mathcal{H}(\boldsymbol{\pi}_\theta)$ of policy function $\boldsymbol{\pi}_\theta$ is added to the policy loss (Equation (2)) to boost exploration, a commonly-used technique in RL literature. The overall learning loss has become

$$L(\theta,\phi)=\sum_{i=1}^{n}\mathcal{L}^i(\theta)+\mathcal{L}^i(\phi)+\lambda_{entropy}H(\boldsymbol{\pi}_\theta)\tag{5}$$

where $\lambda_{entropy}$ determines the scale of the entropy term and a larger value indicates higher exploration. The pseudocode of PPO training is shown in Algorithm 1.

Algorithm 1. PPO training workflow

Input: Initial policy parameters θ_0 of policy Π_θ and value function parameters ϕ_0 of value function V_ϕ ;

for $k=0,1,2,\cdots,K$ **do**

Pull batches of trajectories $\mathcal{B}_k=\{\tau_i\}_{i=1}^b$ collected by policy $\pi_k=\pi(\theta_k)$ from the remote data server;

Compute GAE return $\{\widehat{V}_t^i\}_{i=1}^b$ and advantage estimate $\{\widehat{A}_i\}_{i=1}^b$ based on the current value function V_{ϕ_k};

Update the policy by minimizing the policy loss $\mathcal{L}(\theta_k)$ (Equation (2)) via stochastic gradient ascent with Adam, receive new policy parameter θ_{k+1};

Update the value function by minimizing the value loss $\mathcal{L}(\phi_k)$ (Equation (4)) via stochastic gradient ascent with Adam, receive new value function parameter ϕ_{k+1};

end

Return a trained policy parameter θ_K and trained value function parameter ϕ_K

In practice, since players are homogeneous where the observation and action space are identical, we set up a pair of policy function (actor) and value function (critic) and share parameters θ, ϕ across all players. During training, we apply the Adam[29] optimizer with a learning rate of 5×10^{-4} and a clipping parameter of $\varepsilon=0.2$. The detailed hyperparameter settings can be found in Appendix A.4.

3.4 Population-based training

We use IPPO as the backbone MARL algorithm to learn policies through reward maximization. To further improve the general abilities of the policy and make it less exploitable when facing strong opponents, we also implement population-based training which maintains a population of policies for one player/team in the game and updates the population based on genetic or evolutionary algorithms. In particular, we leverage the self-play mechanism[30] to explore the policy space where an agent learns stronger tactics by competing with itself. The improved policies are added to the population pool and can potentially be chosen as the opponent candidate in the future. As expected, competing with incrementally stronger opponents enhances the general abilities of the population[31].

We implement two population-based training methods, policy-space response oracles (PSRO)[30] which we utilize to rapidly build up a population of football AI at the early stage and league training[7] at the late stage to improve general skills.

Policy-space response oracle. In a two-team normal-form game[31] denoted as (Π, U). $\Pi=(\Pi_1,\Pi_2)$ denotes the set of policy profiles containing strategies of both teams and $U:\Pi\to R$ is the utility function that evaluates the utility of a particular policy profile. A payoff matrix $\mathcal{M}$ has every entry representing the utility of a particular policy profile (π_i,π_j) and can be expanded once a new policy profile has been evaluated. The workflow of a PSRO trial is shown in Fig. 1 and is summarized as follows:

1) Initialize the population with a policy set Π^{init} and compute initial payoff table $\mathcal{M}$;

2) At each generation, select the mix-strategy opponent Π^{mix} by computing the Nash equilibrium[33] on payoff table $\mathcal{M}$;

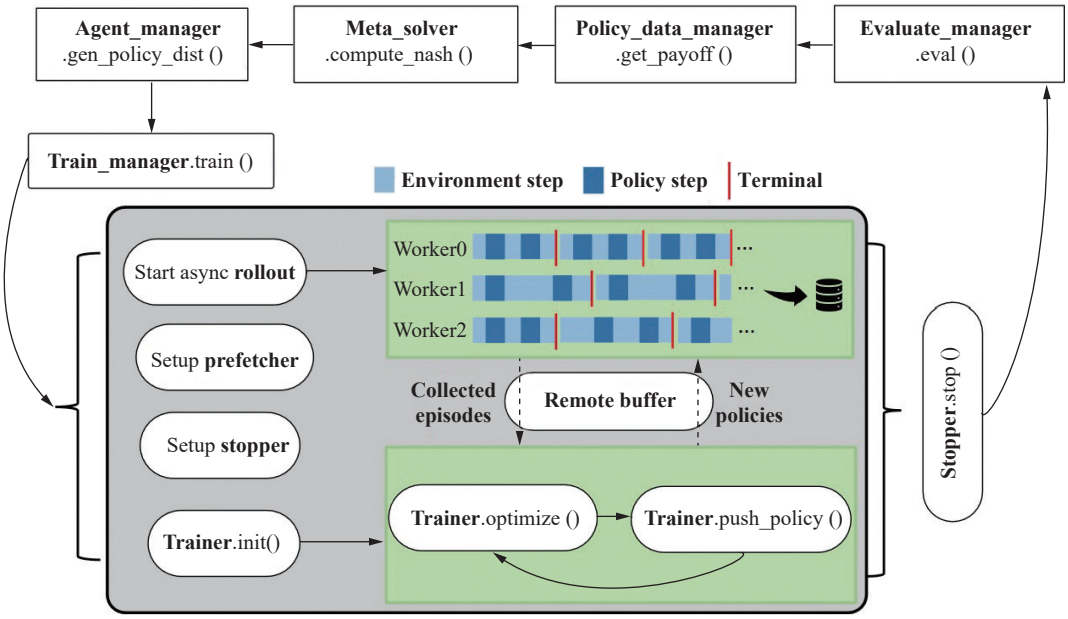


Fig. 1 Workflow of a PSRO example in the Light MALib framework. The loop begins from an evaluation task (top right rectangle) and is followed by the construction of meta-strategies. The training manager activates trainers and generates rollout tasks with the given meta-strategies. The trainers iteratively carry out training and update policy to the remote buffer, while receiving the collected samples coming from the asynchronous rollout worker (similar to IMPALA[32]) through the buffer as well. The rollout worker renews its policy periodically. The process terminates according to some stopping criteria and proceeds to the next generation where the evaluation begins again.

3) Train a multi-agent policy controlling multiple players best responding to a mix-strategy opponent Π^{mix} and obtain the trained policy set $\Pi^{br} = BR\left(\Pi^{mix}\right)$;

4) Add the new policy Π^{br} for each team to the population and the expected utility obtained by the simulation to the payoff table $\mathcal{M}$;

5) Move to step 2.

Since in GRF football games, both teams are symmetric, two sides of the player can follow strategies from a shared population pool. The pseudocode of a PSRO trial is shown in Algorithm 2 where we add the benchmark AI (built-in AI) to the population.

Algorithm 2. PSRO with benchmark AI included

Input: A population Ψ_β with size of β, a policy pool Π with random policies π_{θ_1}, Benchmark AI π_b, and other extra policies π_e;

Randomly select two policies π_l and π_r as the initial policy of two teams (left team & right team);

Initialize α as the learning rate, γ as the discounted factor, $\mathcal{G}$ as the max generation;

Initialize $\mathcal{M}_{\beta\times\beta}$ as the payoff matrix for the meta game compute exp. utilities for each policy π_{i}, where $i \in \{1, \cdots, \beta\}$;

for $j = 1, 2, \cdots, \mathcal{G}$ **do**

  if $j == 1$ **then**

    set the opponent π_o with Benchmark AI;

  end

  else if $j > 1$ **then**

    set the opponent π_o with a probability p sampled from Nash equilibrium policies of the spayoff matrix $\mathcal{M}$;

  end

  Train a multi-agent policy π_j as the best response opposite to π_o with IPPO;

  Push π_j to the population Ψ_β, and update payoff matrix $\mathcal{M}$;

End

Return a set of best policies with different Elo rates

Fig. 2 shows schematic plots of two PSRO variants that differ mainly in best-response policy training (Step 3). The first implementation of PSRO approximates a

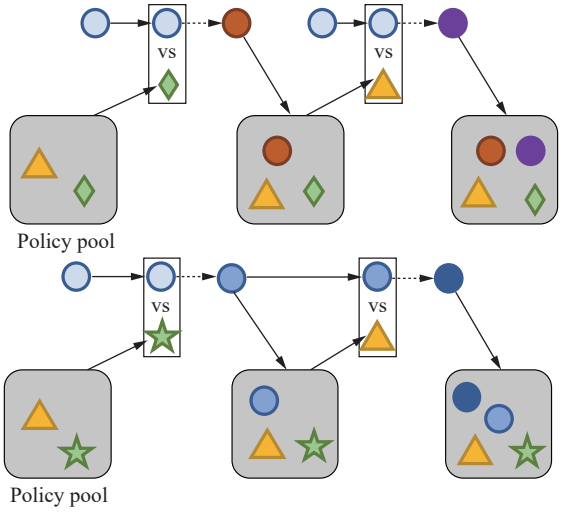


Fig. 2 Top: PSRO with initializations and bottom: PSRO with inherits. Objects with various shapes indicate policies with different tactics.

new best-response policy from scratch at each round. This often means that the policy needs to explore the game and gradually acquire basic skills such as dribbling the ball towards the opponent′s goal repeatedly. A variant named PSRO with inherits inherit the best-response policy from the last round and continues learning against the new opponents in the current round. This largely saves computational time but suffers from the similar style-of-play among the trained policies.

In our implementation, we adopt the second version of PSRO and address the issue of limited diversity by setting up diverse reward shaping.

League training. League training is similar to PSRO but has a more complex framework that has been used to beat professional human players in StarCraft[7]. In league training, a main-agent is trained constantly to keep improving its ability (similar to PSRO with inherits) and the opponents are sampled following prioritize fictitious self-play[7] (PFSP) which selects the opponents it cannot beat based on win rate. Thus, the goal of the main agent is to beat everyone in the population rather than a game theoretical subset of policies, enlarging the scope of opponent style-of-play and improving general abilities. In addition, an exploiter policy is reset at every generation and trained against the newest main agent only. Its job is to exploit the main-agent′s strategic weakness and help to fix it by adding a copy of itself to the population. The exploiter can also potentially discover a new style-of-play as it does not inherit or only inherits from a general pre-trained policy and has large space for evolving to counter the opponent. Furthermore, we drop the league exploiter to ease the computational burden.

The pseudocode of a league training trial is shown in Algorithm 3. In practice, the training of exploiter-agents runs in parallel with the training of the main-agent.

Algorithm 3. League training

Input: A population Ψ_β with size of β, a main-agent policy $\pi_{m,0}$;

Initialize α as the learning rate, γ as the discounted factor, $\mathcal{G}$ as the max generation;

Initialize $\mathcal{M}_{\beta\times\beta}$ as the payoff matrix for the meta game compute exp. utilities for each policy π_i, where $i \in \{1, \cdots, \beta\}$;

Result: A main-agent policy;

for $j = 1, 2, \cdots, \mathcal{G}$ **do**

  Set the opponent π_o with a probability p following PFSP;

  Train the current main-agent policy $\pi_{m,j}$ as the best response opposite to π_o with IPPO;

  Push the trained policy into the population Ψ_β and update payoff matrix $\mathcal{M}$;

  If main-agent finishes training **then**

    Initialize an exploiter-agent policy $\pi_{e,j}$;

    Set the opponent $\pi_{m,j}$;

    Train the exploiter-agent policy $\Pi_{e,j}$ as the best response opposite to $\pi_{m,j}$ with IPPO;

    Push the trained policy to the population Ψ_β and update payoff matrix $\mathcal{M}$;

  end

End

Return current main-agent policy $\pi_{m,\mathcal{G}}$

In Section 5, we present empirical results of MARL training using the IPPO algorithm against built-in AI with difficulty 1.0 in GRF 11-vs-11 full-game scenarios. In Section 6, we discuss how to use a population-based training framework to improve the general abilities of the policies and build strong football AI.

4 Distributed framework: Light-MALib

To ensure consistency in the evaluation of MARL training, we open-source our Light-MALib (L-MALib) framework. As the name suggests, L-MALib is a simplified version of the MALib[3] codebase that uses Ray as a backend and offers population-based training pipelines (e.g., PSRO). L-MALib follows the design of actor-based RL systems that execute algorithms through message passing between a set of actors deployed on worker nodes. Similar to MALib, L-MALib possesses a high-level agent/evaluator/learner abstraction but simplifies the workflow and optimizes efficiency to cater to large-scale training on GRF multi-agent scenarios. More comparisons between L-MALib and MALib can be found in Appendix A.2.

4.1 L-MALib workflows

Fig. 2 provides an overview of a PSRO trial using L-MALib in an evaluation-training/rollout-evaluation loop. The evaluation manager is responsible for simulation between policies in the pool. The policy data manager and the meta-solver analyse the results and the agent manager generates the policy distribution for rollout tasks and the trainable policy id for training tasks. Inside the training manager, rollout and training tasks are executed asynchronously. The rollout workers receive a new policy from the remote buffer and return the collected episodes. The training workers constantly fetch samples from the buffer and renew the policy copy periodically.

On GRF specifically, L-MALib allows training MARL algorithms against built-in AI and a group of trained policies, as well as evolving a policy population by iterating between training and opponent selection. In addition to GRF-related content, L-MALib also provides a smaller-scale multi-agent RL environment benchmark for framework testing such as Leduc Poker[33], Kuhn Poker[31] and ConnectFour[34].

4.2 GRF toolkits

To better evaluate algorithmic performance on foot-

ball, we have also designed a football event detection data structure and developed a game debugger for detailed analysis per game. The new data structure as shown in Fig. 3 abstracts every change in ball ownership and records the involved players′ status, such as the player who passes the ball and the player who receives it in a ball passing event. This enables us to accurately locate every game event such as a good assist or a bad pass and correspondingly assign individual credits. The game debugger loads from the official dump file of each game replay and provides elaborate 2D visualization of the game data as well as trajectory analysis for each individual player. This is beneficial for strategic analysis and helps identify weaknesses.

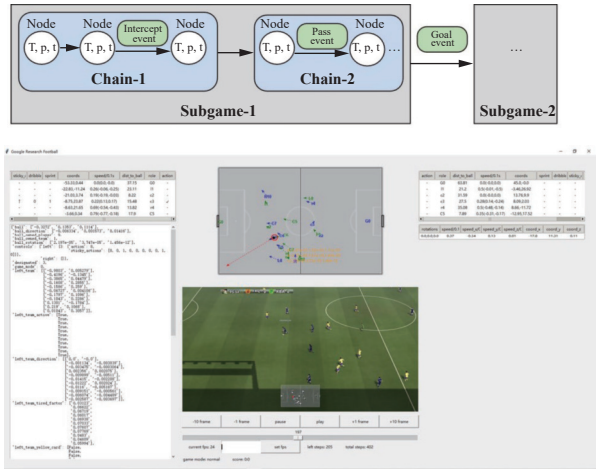


Fig. 3 Game graph and game debugger designed for GRF

In brief, we present L-MALib, a population MARL distributed framework that inherits structural design from MALib but is specifically optimized for experiments on Google research football. We implement independent proximal policy optimization (IPPO) algorithms and test them on 11-vs-11 multi-agent full-game scenarios.

4.3 Time cost estimate

Light-MALib speeds up the training phase by running environment rollouts and policy updates in a distributed and asynchronous manner. In addition, as we treat all players homogeneously by parameter-sharing, only a pair of actors and critics is maintained for each policy and hence the inference and update of networks can be performed in a single batch. During decentralized execution in a real game, each player has a copy of the trained policy and makes decisions based on his own observations. Specifically, at each decision step:

1) Each agent receives its own observation and encodes it into a high-dimensional feature vector;

2) Each agent feeds the feature vector into its actor network and decodes an action. The time cost is roughly the time of forward-passing of three-layer neural networks in our setting;

3) All actions are then collected and passed to the game engine. Finally, an environment step is executed and new observations are returned.

The time cost and relative proportion of each substep during rollout and training are shown in Fig. 4. During policy execution, the GRF game engine takes 73% of the time and the batched policy inference only occupies 9.5% of the runtime. The maximum running speed is roughly 71 frames per second and on a 3 000-step episode, the estimated time cost is approximately 1 minute. Thus, our AI can run in a real-time manner when deployed. During policy training, most of the time is spent on GPU loading and loss computation. Practically, the total runtime of our experiments is largely restricted by the policy execution and particularly the engine processing.

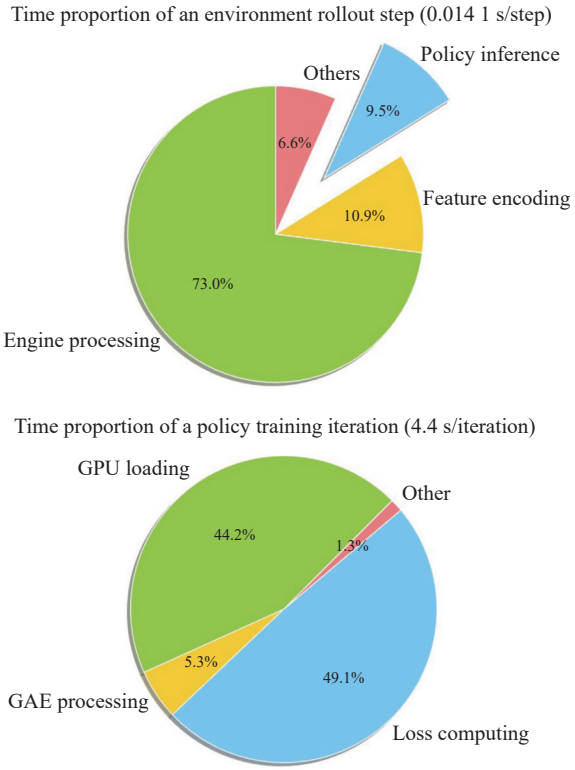


Fig. 4 Time proportion plot of environment rollouts (Top) and policy training iterations (Bottom). An environment rollout step takes approximately 0.014 seconds and most of the running time is used for engine processing. A training iteration takes approximately 5 seconds and most of the running time is used for GPU loading and loss computing.

5 Empirical results of MARL training

Through our experiments on GRF, we found several factors that are critical to performance such as the training resources, feature encoder, discount factor and reward shaping. In this section, we evaluate independent PPO with the commonly-used setting on 11-vs-11 multi-agent scenarios. We apply only mild tuning and empiric-

ally investigate the following questions:

1) *How should feature encodings be chosen?*

2) *To what extent does discount factor γ affect performance under such a long-horizon task?*

3) *How does reward shaping affect performance in failure cases?*

4) *How many computational resources are required to achieve good performance within a reasonable amount of time?*

5) *How does the obtained policy behave?*

The detailed experimental settings can be found in Appendix A.4, including the feature encoding, action masking, reward shaping and hyperparameter settings. All the empirical results are obtained from the following hardware settings:

***System*#1:** A 128-core computing node with NVIDIA RTX3090 GPU (24 G memory);

***System*#2**: A 256-core computing node with NVIDIA A100 GPU (40 G memory);

***System*#3**: A 300-core computing node with 8x NVIDIA 3 080 GPUs (80 G memory in total).

5.1 Performance on different feature encoding

We compare the performance of three different feature encodings: official simple115_v2, encoder_basic and encoder_enhanced as discussed in Appendix A.4. As the original simple115_v2 encoder did not use action masking, we implement it with an action mask for a fair comparison. The results are shown in Fig. 5. The vectorized simple115_v2 performs terribly, indicating the need for extra information in the feature embedding. Encoder_basic performs the best by introducing states of closest teammates and opponents. The encoder_enhanced features have slower convergence. This is possibly due to the insignificance of extra information about the match states and off-side position when competing with built-in AI.

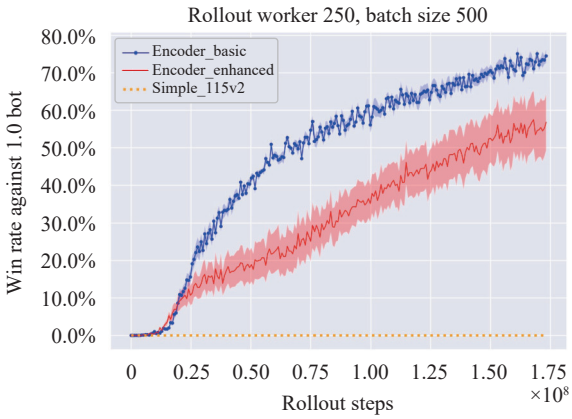


Fig. 5 Rollout win rate against the 1.0 hard bot with the official simple115_v2 (dashed line), encoder_basic (solid line with dot marker) and encoder_enhanced (solid line) feature encoders

However, when training in a population-based style, we resort to an enhanced encoder as we need more complicated features to counter stronger and more diverse opponents, such as using offside information to avoid off-side trapping.

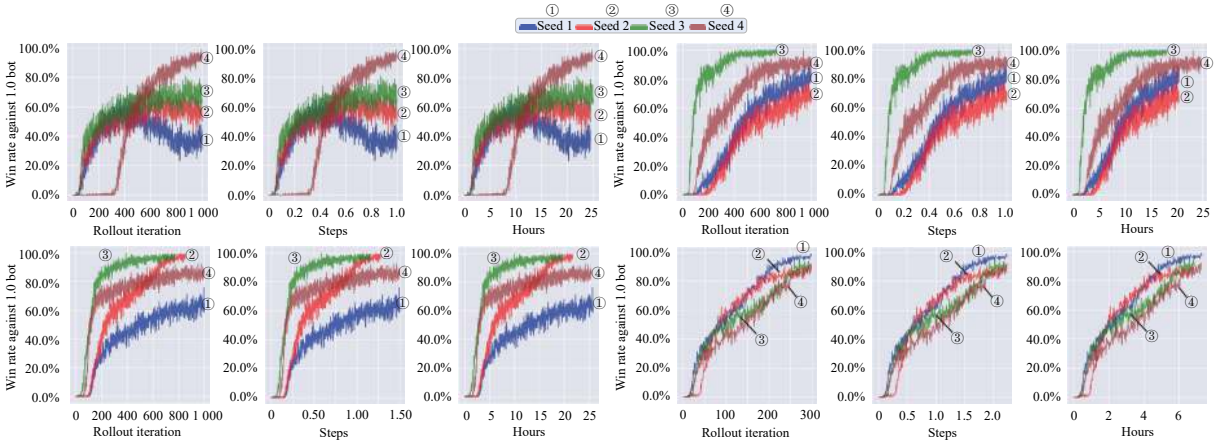
5.2 Performance on different discount factors and failure cases

Our experiments also demonstrate that in 11-vs-11 multi-agent scenarios, a discount factor γ equal to 1 provides more stable training performance as illustrated in Fig. 6, where $\gamma = 1$ indicates faster convergence and higher stability. The performance in each seed can be found in Fig. 7, where seed 4 is hard to learn with a lower discount factor. To explore why this happened, we evaluate the learned state-value function V_ϕ on a testing trajectory and the results are drawn in Fig. 8. We see that policies trained with $\gamma = 0.995$, 0.99 have a large variance V_ϕ across time steps and show greater aversion to goal loss as well as states that can potentially lead to it[2]. This might be due to the large TD error during bootstrapping as illustrated. The large TD error comes from the large scale in V due to the non-zero discount factor and long-horizon which has also been noted in [28]. As a result, a large value in δ results in high variance in GAE returns across time (according to (3)), which further leads to high variance of the updated state-value estimate V. This renders the behavior of the actor pessimistic since the collected samples at the early training stage are often unrewarding over which V shows great aversion. Additionally, in Fig. 9, we evaluate the same statistics on checkpoint models of both successful and failed policies

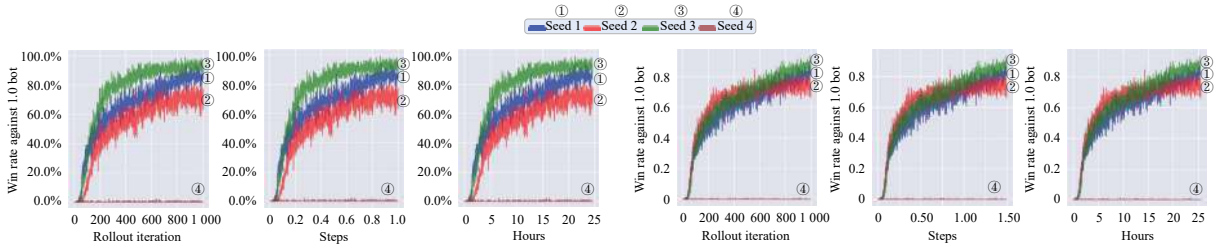


Fig. 6 Rollout win rate against 1.0 hard bot across different discount factors. The solid line with dot markers is at discount factor 1, the line with cross markers is at 0.995 and the solid line without markers is at 0.99. Results are averaged over four random seeds and the shaded area shows the standard error.

[2] Reviewing the game replay, we found that the sharp drop in state-value estimates at the time-step where no goal/lose goal happens is a consequence of opponent missing a shot.



(a) Discount factor $\gamma = 1$



(b) Discount factor $\gamma = 0.995$

Fig. 7 Performance in different seeds (1, 2, 3 & 4) at discount factors of 1 and 0.995: (a) Configurations from top to bottom denoted in the form [rollout-worker, batch-size, discount-value] are: [100, 100, 1]; [100, 200, 1]; [150, 300, 1]; [250, 500, 1]; (b) The upper is [100, 200, 0.995] and the lower is [150, 300, 0.995]. (①②③④: Seed 1, 2, 3 & 4.)

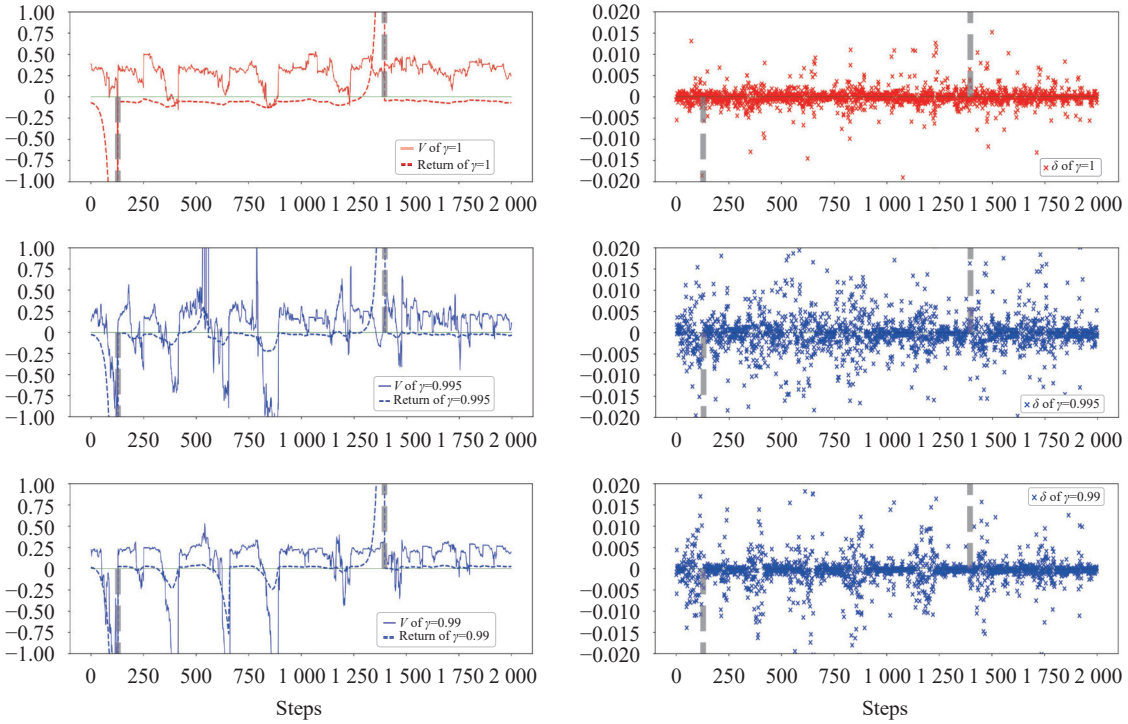


Fig. 8 State-value estimate (left) and TD error (right) of policy trained with discount factors of 1 (top), 0.995 (mid) and 0.99 (bottom) on a testing trajectory. The solid line indicates the state-value estimate V and the dashed line indicates the GAE return across each time-step. The thick grey vertical bar represents the reward of goal (positive) and loss of goal (negative).

throughout training. The policy that learns to defeat bots at $\gamma = 1$ manages to withhold its extreme aversion over unrewarding events and keep the value of δ small as the training proceeds, whereas the policy with $\gamma = 0.995$

founds it hard. Furthermore, Fig. 10 compares the values of V and verifies that models trained with discount factors of 0.995 and 0.99 enlarge the scale of V and make it pessimistic incrementally throughout training. Value clipping does not alleviate this issue. Therefore, we recommend using a high value of discount factor in the 11-vs-11 full-game scenario.

Reward shaping in failure cases. Previously we demonstrated that different discount factors result in different stabilities, and small values may lead to pessimistic behavior. Here, we fix the instability by reducing the negative rewards and investigate how it helps improve performance in failure cases. In Fig. 11, we compare the performance of different reward shaping weighting at $\gamma = 0.995$, $seed = 4$, where zero-sum reward weighting (+ 1 for making a goal and −1 for losing a goal) does not work. After scaling down the weights of negative rewards assigned to goal losing events, the policy trained with $\gamma = 0.995$ manager to beat the bot. The fastest convergence occurs when the negative reward weighting is 0.2. This indicates that the negative rewards in GAE can potentially cause instability in long-horizon.

Meanwhile, this can also be due to the purely-offence style of plays from the trained policy where charging forwards is efficient to cause opponent offside and retake the ball ownership (as shown by the high win rate orange curve in Fig. 11, where only positive rewards are considered).

5.3 Performance on different computational resources

Having determined a good feature encoder and the discount value, we evaluate the L-MALib performance of different resource configurations. The results can serve as a reference for researchers interested in 11-vs-11 scenarios. Note that these experiments do not indicate parallelism performance but only provide a clue on how to configure training given available computing resources (on a single GPU machine or multi-GPU multi-node cluster). The factors we aim to test are as follows:

1) Number of rollout workers: number of parallel tasks that collect and return samples from environments. Each rollout work holds one environment task by default. The choices are [100(#1), 150(#2), 250(#3)][3].

2) Batch size: number of data sampled from the data server in each training iteration. The choices are [100(#1), 200(#1), 300(#2), 500(#3)].

We choose encoder_basic feature encoders and $\gamma = 1$. We run rollouts and model training on CPU and GPU respectively. The number of rollout workers determines the efficiency of sample generating on CPU machines, and the batch size determines the amount of sample used to compute the gradient per training iteration on GPU machines. We run experiments against the 1.0 hard bot with the same hyperparameter settings but different numbers of rollout workers and batch sizes on three systems. Fig. 12 illustrates the number of rollout iterations required to reach a good average win rate, where we see that GRF is a very resource-consuming testbed for IPPO, and within the capability of our machine, the performance becomes stably better given more resource allocation. In Fig. 13, we also observe that our implemented algorithm is able to provide robust performance in a large scale setting.

Fig. 7 explicitly shows the corresponding time performance at each seed, and Table 2 lists the averaged time cost at a particular win rate. On Systems #1 and #2, our training framework costs approximately 15.7 hours to beat built-in AI with difficulty 1.0 from random initialization, and this time cost can be reduced to less than 5 hours, which is acceptable to use even on small-scaled clusters (such as System#1).

Table 2 Averaged time cost at particular win rates (20%, 40%, 60%, 80%)

	20%	40%	60%	80%
$RW = 100$, BZ $= 100$ (#1)	3.7 h	5.1 h	12.7 h	NA
$RW = 100$, BZ $= 200$ (#1)	4.2 h	6.1 h	9.7 h	15.7 h
$RW = 150$, BZ $= 300$ (#2)	3.1 h	5 h	8.8 h	15.7 h
$RW = 250$, BZ $= 500$ (#3)	**0.6 h**	**1.6 h**	**3.3 h**	**4.9 h**

In brief, the carefully-designed feature encoders provide extra information and enable faster convergence. The high value of the discount factor ensures the stability of training and the larger computational resources offer higher efficiency and more robustness.

5.4 Charging forward is efficient to beat the 1.0 hard bot

Although we have achieved a fairly high win rate against the 1.0 hard bot, the style-of-play of all of our trained policies converges to a fixed offense and defense pattern demonstrated by snapshots of game replay in Fig. 14. On defense, all players controlled by the trained policy will move forwards towards the opponent′s half to cause offside and dribble the ball all the way to the penalty area to make a shoot while attacking. This strategy works against bots surprisingly well since the built-in AI logic designed for hard bots sometimes does not recognize offside cases and has considerable defensive vulnerability. However, this charging strategy is also easily exploitable as it overfits to the glitch in bot.

In other words, being able to beat the hard bot does not reveal strong general skills and we ought to use a more complicated framework, such as population-based training, to build general football AI.

[3] #1, #2, #3 represent experiments run on systems #1, #2, #3, respectively.

6 Empirical results of population-based training

In Section 3.4, we have discussed our implemented population-based training methods. In practice, we build a general football AI through the following procedures:

Stage 1. Rapidly building up a pool of policies that are generally not strong but strategically diverse using diverse reward shaping;

Stage 2. Use the policy pool as diverse initializations and improve basics skills by carefully choosing opponents;

Stage 3. Gather representational policies and adopt a more complex evolutionary framework to improve general abilities.

6.1 Stage one: Increase the reward diversity

We first handcraft K skills in football games including offense and defense with different reward shaping functions. Then we train K populations using PSRO with inherits with K sets of reward functions $r_i(s_t, a_t)_{i=1}^K$. The best team policy with highest Elo rate in each trial would be selected as a representational policy at this particular reward shaping scheme. The detailed reward shaping and algorithm are shown in Appendix A.4.

Note that these K number of policies might not be generally strong since tuning reward shaping towards a sophisticated football strategy is extremely hard and computationally expensive. Thus, at this stage, we are not concerned about the strength of each policy as they are already the best along their evolutionary pathway, but the strategic diversity they express. For example, a trial using players′ distance reward could learn a policy with a high-pressuring strategy while an active defensive reward shaping scheme could offer policies with stronger defense. Expectedly, we will have access to a set of representational policies with distinct strategical styles when stage one ends. However, in practice we encounter a failure case where we include built-in AI in the population of a PSRO trial. The trained policy inevitably sticks to a fixed strategy regardless of the reward-shaping scheme.

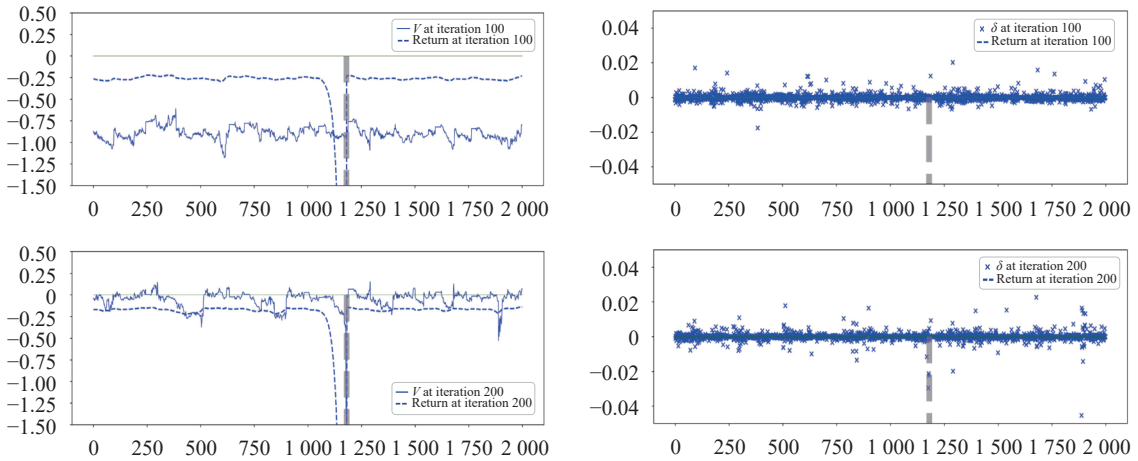
6.2 Fail case: Experiments with built-in AI included

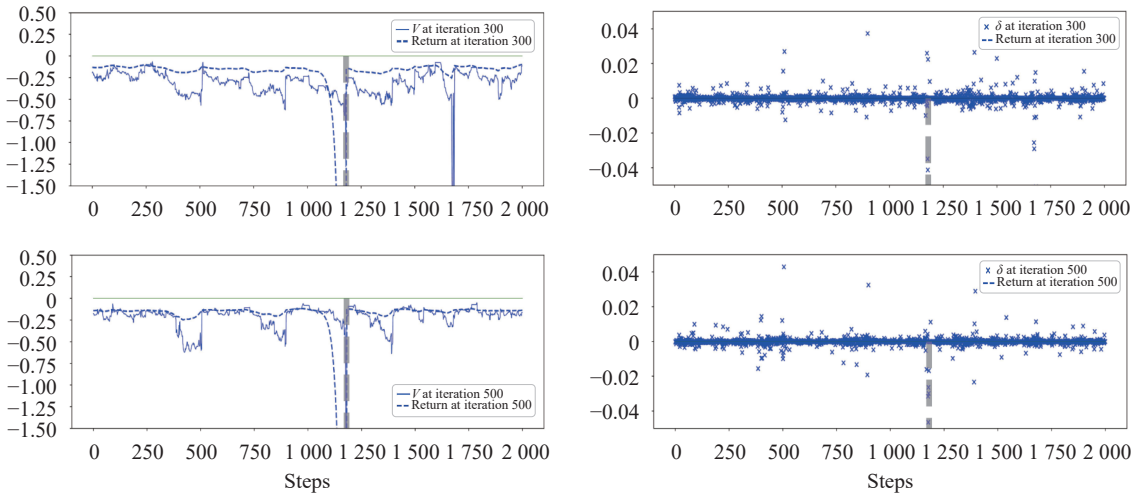
During our experiment, we found that adding built-in AI into the initial population can restrict the playing style of the resulting policy to a large extent. In Fig. 15, we present the results of a PSRO run with a specific reward-shaping scheme. We adopt PSRO with inherits and for each generation, we stop training when the average win rate has exceeded 90% or the maximum number of rollout iterations has been reached. Built-in AI is included in the initial population. We apply the reward factors shown in Table 3 with the following weighting:

$$5 \times (\text{team goal/lose goal}) + 0.2 \times (\text{individual goal and assist}) + 0.05 \times (\text{individual lose ball}) + 0.01 \times (\text{minimum distance}).$$

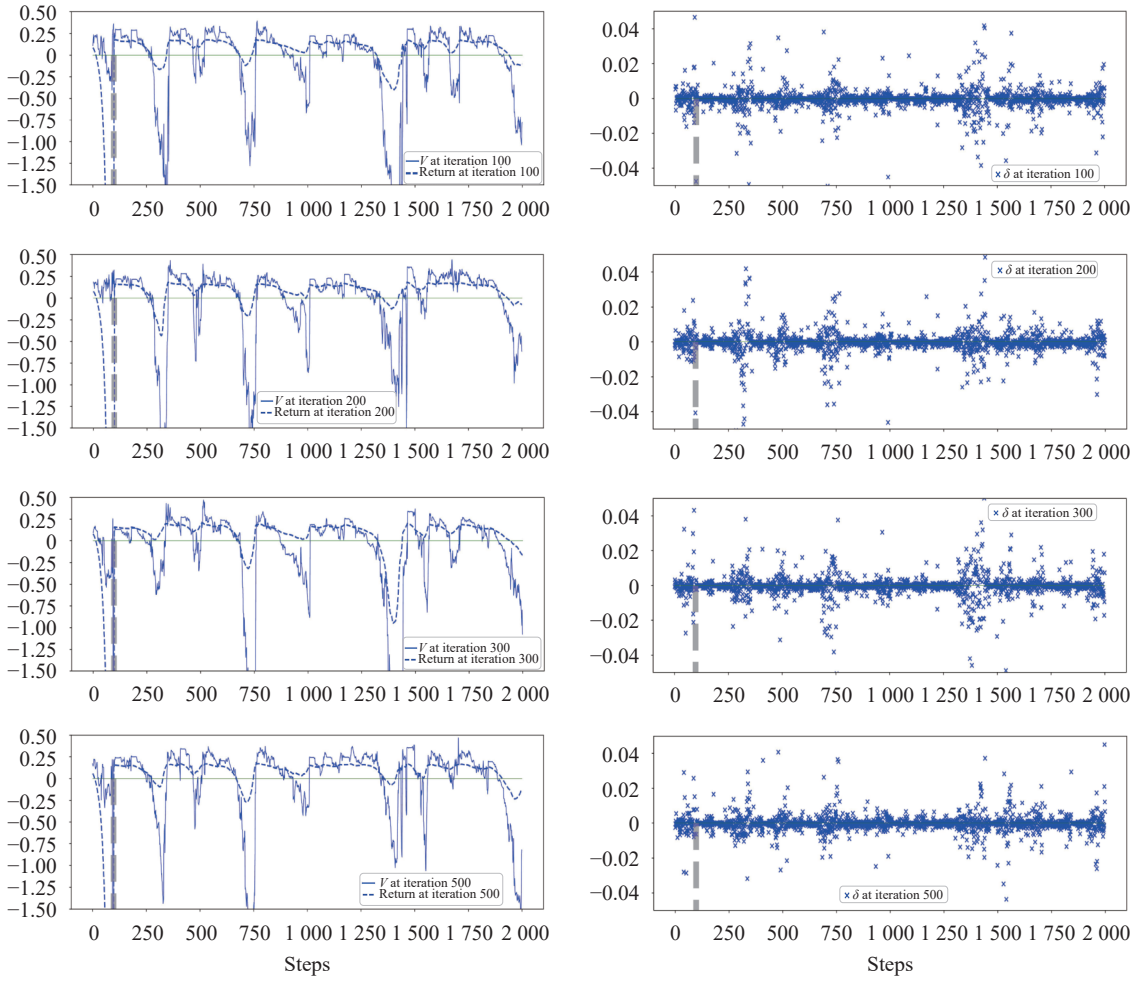
Both the meta-strategy distribution and payoff table show the existence of non-transitivity[35] in this PSRO trial, which means that a policy that can beat another policy that beats the bot, loses to the bot. By reviewing the replay of the game, we found that the newest policy still has the same style-of-play as the policy in the first round which learns to beat the bot. A snapshot of the game replay is presented in Fig. 16, where we see both teams standing at the opponent′s penalty area and showing no defensive move. This is because the policy in the first round learns the charging style-of-play which we have mentioned in Section 5.4 and the following generation inherits this strategy and only updates slightly which is sufficient to beat the past, such as improving shooting accuracy or dribbling.

At the same time, a change in playing style can result in a large drop in the win rate against built-in AI since such a charging strategy is based on the strategic loophole that exists in built-in logic and any deviation from this strategy will be “corrected” by gradient descent. In other words, the charging playing style is a local





(a) State-value estimate V (Left) and TD error δ (Right) from checkpoint models that beat the bots.



(b) State-value estimate V (Left) and TD error δ (Right) from checkpoint models that fail to beat the bots.

Fig. 9 State-value estimate and TD error evaluation on a testing trajectory from (a) checkpoint models that beat the bot and (b) checkpoint models that fail to beat the bots. As the training proceeds, the models in (a) manage to learn a stable V while the models in (b) experience large TD errors and learn a pessimistic V which shows great aversion over unrewarding states.

minima and is hard to escape from it. To verify this, we select policies main_16 and main_17 in Fig. 15. Main_16 gets beaten by the built-in AI (shown on payoff) but at generation 17, the converged policy main_17 beats the built-in AI again since the mix-strategy of the opponent contains both bot and the past (shown on meta-strategy

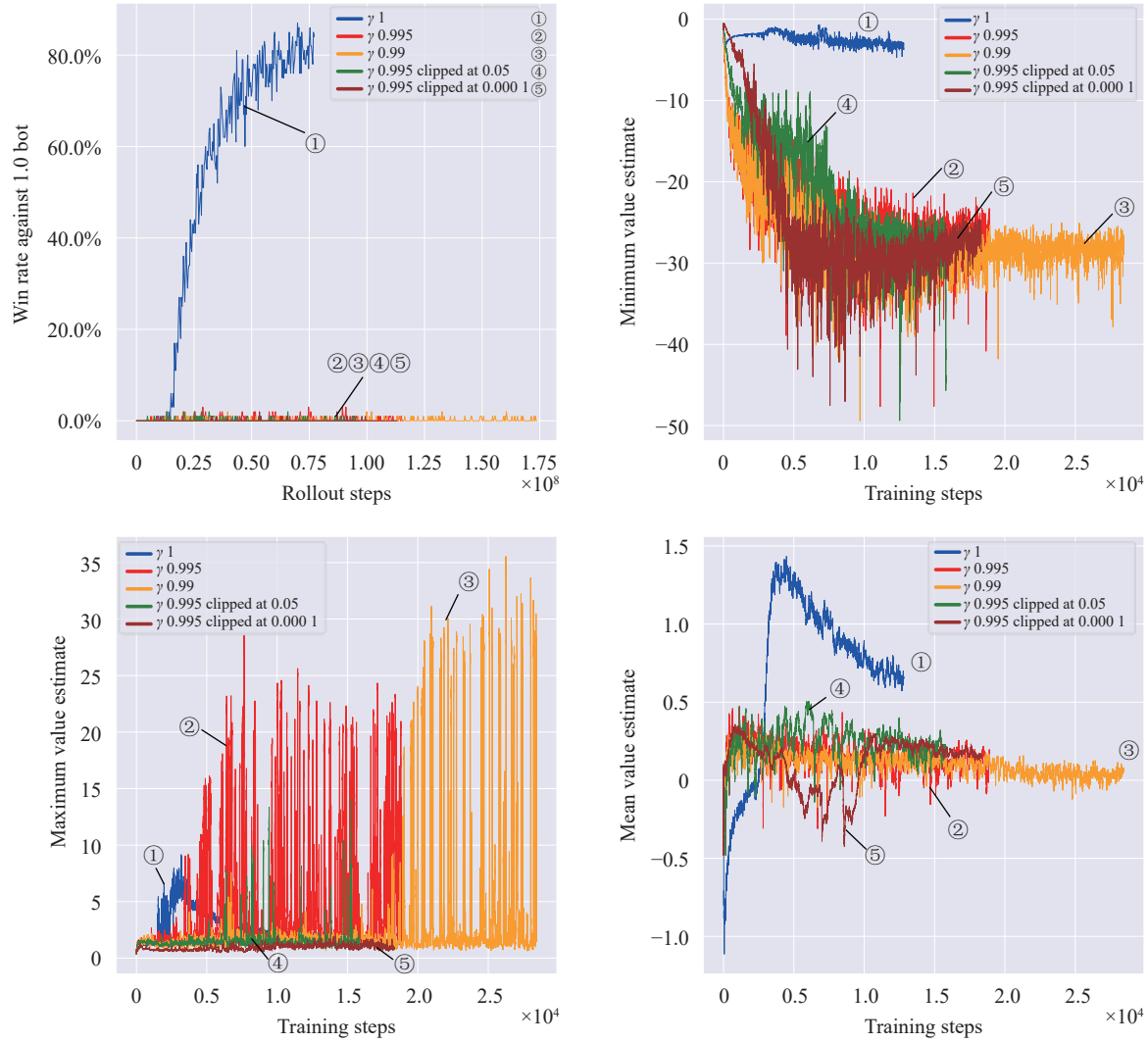


Fig. 10 Win rate and state-value estimate throughout training at different discount values and clipping thresholds (①: $\gamma = 1$; ②: $\gamma = 0.995$; ③: $\gamma = 0.99$; ④: $\gamma = 0.995$ with a 0.05 clipping threshold; ⑤: $\gamma = 0.995$ with a 0.000 1 clipping threshold). All settings at $\gamma = 0.995$ cannot beat the bots and have large scales on V. Value clipping cannot alleviate such issues.

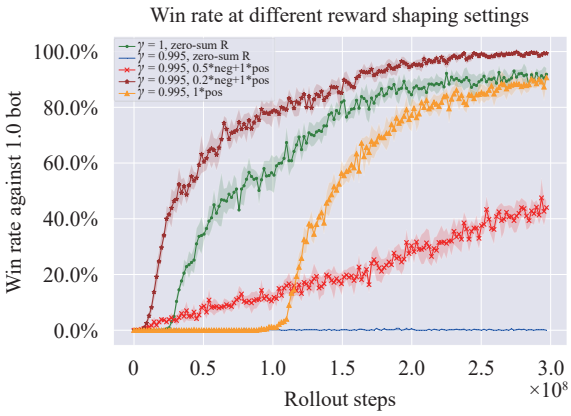


Fig. 11 Comparing the performance of different weights in reward shaping under random seed failure. The curves have been smoothed.

distribution). We look at the game replay from both policies and take a snapshot each at a defense round (shown in Fig. 17). Main_16 shows a defensive move while main_17 gives up defending and takes advantage of the glitch in the bot (not recognizing the offside) to win the game. This happens because learning to defend with specially-designed rewards cannot easily stop the bot's mature attack and therefore the behavior rollbacks to the original strategy cause offside. This indicates that although defensive reward shaping encourages players to play defense, competing with built-in AI can waste such efforts.

In brief, the defensive glitch in built-in AI lures the trained policy to learn the strategy of charging while the strong general skills of built-in AI ensure that the trained policy does not deviate from this extreme playing style. As a result, we exclude built-in AI from the opponent pool and add policies with the highest ELO rate from other PSRO trials to the current initial population. This helps construct a population incrementally with policies that are not generally strong, but strategically diverse. In

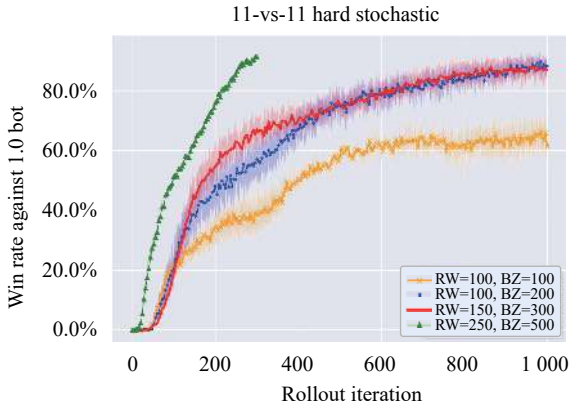


Fig. 12 Rollout win rate against a 1.0 hard bot across different rollout-batches configurations. The results are averaged over four random seeds and the shaded area represents the standard error. For each trial, the average time cost is approximately 70 seconds/iteration and the total time cost for 1K iterations is approximately 19 hours. At each rollout iteration, RW environment tasks are running in parallel and $RW \times 3\,000$ (EpisodeLength) steps of samples are collected. During training, $BZ \times 1\,000$ (SampleLength) steps of data are sent for training.

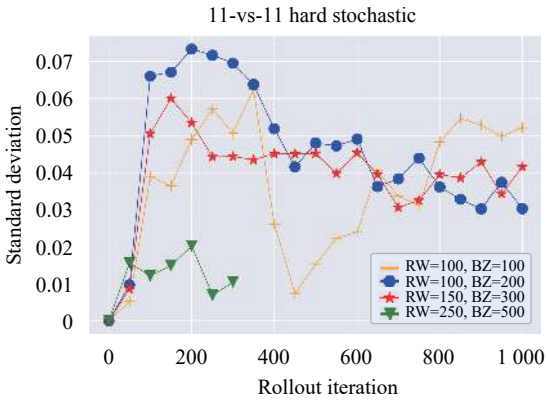


Fig. 13 Standard deviation of win rate against 1.0 hard bot across four random seeds at different resources configuration. Our implemented algorithms get robust performance at large scale.

the next stage, we aim to improve the basic skills of the policies by carefully choosing the initialization and the opponent policies.

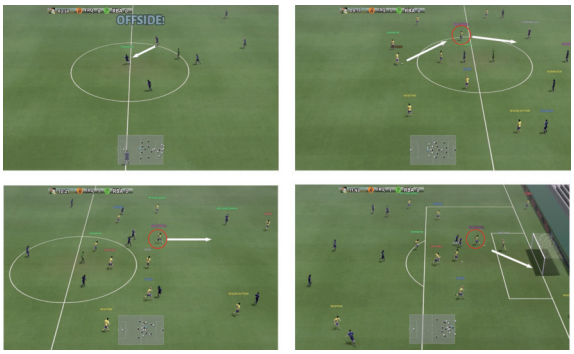


Fig. 14 The behavior of learned policy against hard bot. Hard bot can easily be offside when all of our players are in the right half (top left). We then start the free kick (top right) and pass to one of our teammates who charge forward recklessly (bottom left) and make a shot (bottom right).

6.3 Stage two: Initialization diversity and specific opponent selection

In all of our studies above, we consider how to train effective football policies and build policy populations from scratch. In addition, we provide further guidance on how to constantly improve the policy by using diverse policy initialization and selecting a specific opponent strategic style.

Good initialization is important for training efficiency and policy diversity. Training from scratch is computationally insufficient as the policy needs to acquire basic skills such as passing and shooting, which is difficult in such a sparse and long-horizon scenario, especially when the opponent is already statically sophisticated. Therefore, a warm start at a skillful initialization is essential for good efficiency. Meanwhile, the style of the initialized model is responsible for the style-of-play at convergence of training as we have seen in Section 6.2. Hence, the diversity in the pretrained model is responsible for the diversity of the population. For instance, Han et al.[36] diversify their pretrained models

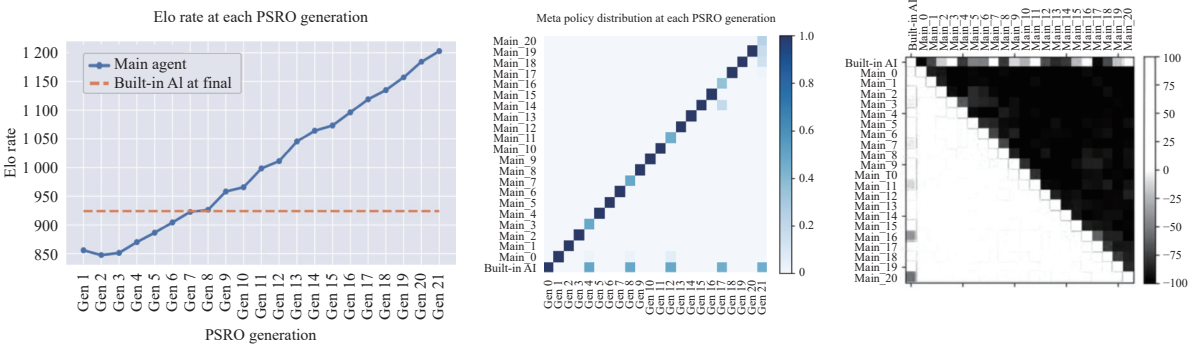


Fig. 15 PSRO on 11-vs-11 multi-agent scenarios. The left is the change in the Elo rate at each generation. The middle is the meta-policy distribution or the opponent strategy distribution at each generation. The right is the payoff table where each entry in the matrix represents the score of the policy on the vertical axis when competing with the policy on the horizontal axis. From the meta-distribution and the payoff table one can observe the non-transitivity in GRF training.

Table 3 Simple reward shaping implemented

Reward factor	Description
Team goal/lose goal	Goal (+ 1) or lose a goal (–1) for all teammates scaled by their roles
Individual goal and assist	Player who goal (+ 1) and player who assist (+ 1)
Individual lose ball	Player who lose ownership of the ball (–1)
Individual gain ball	Player who gain ownership of the ball (+ 1)



Fig. 16 A snapshot of the game replay between two policies during a PSRO run



Fig. 17 Snapshots of a defensive round against bot from policy main_16 (left, team yellow) and main_17 (right, team yellow). The main_16 has two defense players whereas main_17 gives up defending. Main_17 can beat the bot while main_16 cannot.

to ensure the diversity of the exploiter policies. Inspired by this, we use the diverse population obtained from stage one as an initialization pool for the following experiments.

Opponents determine the learned strategy style. Apart from the initialization diversity, we are also concerning about the style evolved during training. After an initialized policy has been set up, one way to guide policy learning is by reward shaping, as discussed in Section 6.1. However, according to our experience, we found that setting up a good reward shaping scheme for sophisticated styles of play is extremely difficult. For example, adding rewards for ball passing can increase the number of passes but most of them are unnecessary and can easily cause turnover. Adding penalties for turnovers will discourage passing and the trade-off is computationally hard to decide.

An easier and straightforward way is to choose the style of the opponent policy for the specific style-of-play we want the trained policy to possess. For example, when competing against a policy with a high-pressured defense, the policy can easily learn to pass the ball even under simple goal and lose goal reward settings. At this stage, we select a good initialization policy and further improve skills by choosing opponents with care. Our released pretrained policies group_pressure and offensive_passer are obtained in this manner. Group_pressure is trained using defensive reward shaping and offensive_passer learns good passing skills by training against group_pressure and some of its checkpoint policies.

To date, we have a population of policies with strategic diversity that are relatively less exploitable compared to stage one. In the final stage, we deploy league training to improve the general abilities of the population.

6.4 Stage three: League training

League training is similar to PSRO but has a more complex framework that has been used to beat professional human players in StarCraft[7]. L-MALib also provides league training framework implementation. In league training, a main-agent is trained constantly to keep improving its ability (similar to the main agent in PSRO with inherits) and the opponents are sampled following prioritize fictitious self-play[7] (PFSP) which selects the opponents it cannot beat based on the win rate. Thus, the goal of the main agent is to beat everyone in the population rather than a game theory subset of policies, enlarging the scope of opponent style-of-play and improving general abilities. In the meantime, an exploiter policy is reset every generation and trained against the newest main agent only. Its job is to exploit the main-agent's strategic weakness and help the main-agent fix it by adding a copy of itself to the population. The exploiter can also potentially discover a new style-of-play as it does not inherit or only inherits from a general pretrained policy and has large space for evolving depending on the opponent. An example of a league training trial is shown

in Fig. 18. In Fig. 18(b), we also provide plots of time cost at later stage of league training for reference purposes.

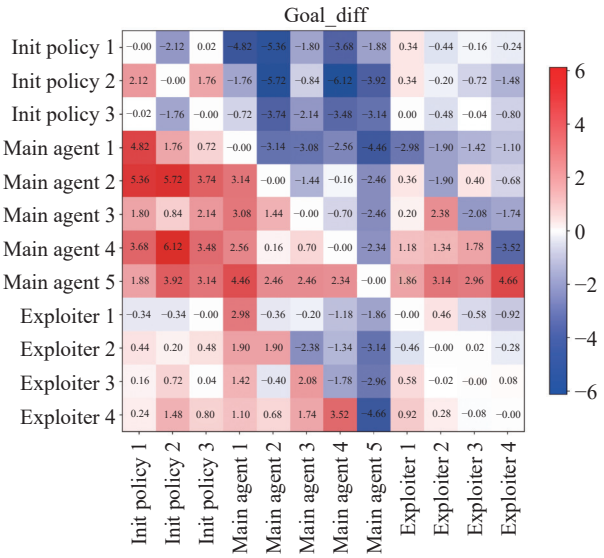
League training can incorporate ideas from both stages one and two. The exploiter can learn offensively and defensively using offense and defense rewards respectively. Each of them exploits the main agent from different strategic aspects. The offensive exploiter examines the main agent's defensive ability and vice versa. The exploiters can also adopt different initialization policies. The opponent selection mechanism of league training also ensures opponent diversity and encourages the main agent to grasp general skills to beat everyone in the population.

We follow the three-stage procedures and obtain general football policies that are strong both offensively and defensively. We release some of the policies with representational style-of-play, serving as good initialization and a strong opponent to combat.

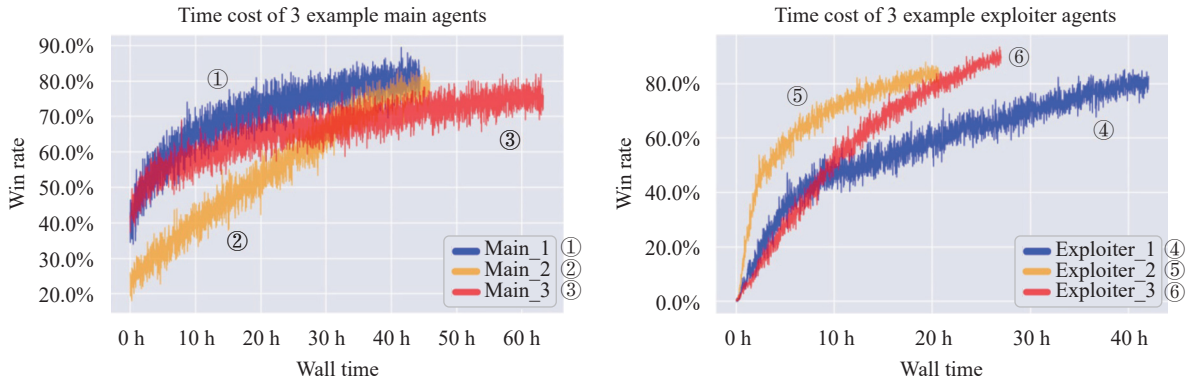
6.5 The trained policies

We release five representative trained-policies with diverse football tactics. Taking inspiration from statistical football tactics research[37, 38], we evaluate their performance from 11 football-specific metrics using position data collected from policy simulations. Those metrics are: Goal, shot, shot accuracy, ball possession, pass, pass accuracy, tackles, interceptions, assist, player movement and team formation. Most statistics are collected from event detection and the team formation is reviewed by weighted team centroid, effective player space, team separateness and player length per width. A radar plot is in Fig. 19.

Weighted team centroid is computed similarly to team



(a) The payoff table of a league training trial



(b) The time cost of 3 example main and exploiter agents at later stage of league training. (①②③: main_agent_1, 2&3; ④⑤⑥: exploiter_agent_1, 2&3)

Fig. 18 (a) The payoff table of a league training trial. The init_policy 1, 2 and 3 are the initial policy populations obtained from stage 2. Each entry represents the goal difference of the policy on the vertical axis against policy on the horizontal axis. The exploiter targets solely the newest main agent and the main agent needs to beat everyone in the population. The exploiter is also initialized from policies at stage 2. We see that the exploiter keeps exploiting the newest main agent and the main agent keeps learning to beat the new exploiter. (b) The time cost plot of 3 example main and exploiter agents at later stages of league training. Each generation of main and exploiter agents takes 1–2 days to reach an 80% win-rate.

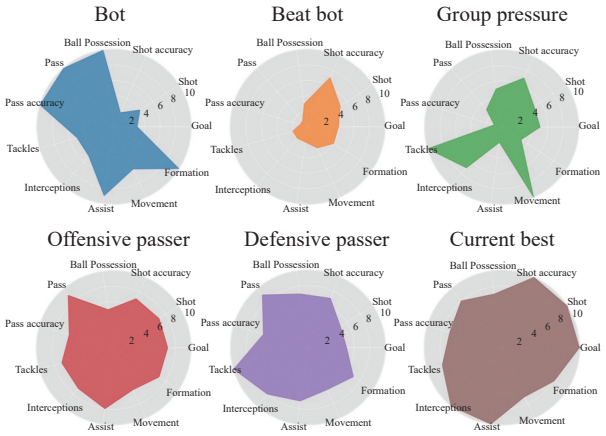


Fig. 19 Skill radar plots of the released policies and the 1.0 hard bot. The statistics are collected from matchup against each other and rescaled to be shown. Group_pressure is obtained from stage 1; offensive_passer is obtained from stage 2; defensive_passer and current_best are from stage 3 league training. We can see that the 3-stage procedure improves general skills of the policies.

centroid but weighted by distance between players and the ball. Effective player space (EPS) is computed as the area of the largest convex hull cover by the players. Team separateness is the sum of distance between each player to its closest opponent. The player length per width is the proportion of pitch length and the pitch width used by the player[37, 38].

Among all released policies, beat_bot is trained solely against the hard bot and exploits glitches in bots. Group_pressure is obtained from defensive reward shaping at stage 1, which exerts high pressure on ball possession. The offensive_passer is obtained from stage 2 by carefully selecting a high pressure policy to encourage ball passing. The others are from league training where defensive_passer is one of the exploiter policies and current_best is the newest main agent. They learn to pass the ball and have better general skills, such as zone defense and cross-from-side tactics. This has shown the effectiveness of our 3-stage procedure in terms of building general football AI. We hope that these benchmark policies can serve as good pretrained models and good opponents to compete with for research purposes.

7 Conclusions

In summary, we present a population-based multi-agent distributed RL framework and target specifically on Google research football testbeds. We provide an empirical study on 11-vs-11 multi-agent full-game scenarios from scratch that are rarely explored and lack open-sourced evaluation. Our experiments show that the parameter-sharing independent PPO is sufficient to beat the bot with 1.0 difficulty but the resulting behavior is highly exploitable. We also present the population-based training pipeline to build strong football AI. All of our code and trained policies are released.

Acknowledgements

This work was supported by the National Natural Science Foundation of China (No. 62206289).

Declarations of conflict of interest

The authors declared that they have no conflicts of interest to this work.

Appendix

A.1 Notation

Table A1 Notations used in this paper

State space	$\mathcal{S}$
Action space	$\mathcal{A}$
Observation space	$\mathcal{O}$
Reward function	$R(\cdot)$
Transition function	$P(\cdot)$
Number of agents	n
Discount value	γ
Time step	t
State at t	s_t
Joint action at t	A_t
Individual action at t	a_t
Observation at t	o_t
Policy function	$\pi(\cdot)$
Policy function parameter	θ
Previous policy function parameter	θ_{old}
Value function	$V(\cdot)$
Value function parameter	φ
Objective function	$\mathcal{J}(\cdot)$
Expectation function	$\mathbb{E}(\cdot)$
Loss function	$\mathcal{L}(\cdot)$
State occupancy distribution	ρ
Advantage function	$\widehat{A}(\cdot)$
Noise parameter	ϵ
Clipping operation	$clip$
GAE lambda	λ
TD error	δ
Entropy function	$\mathcal{H}(\cdot)$
GAE return	$\widehat{V}$
Entropy coefficient	$\lambda_{entropy}$
Set of policy profiles	Π
Utility function	$\mathrm{U}(\cdot,\cdot)$
Compute best response policy	BR
Payoff matrix	$\mathcal{M}$

A.2 L-MALib and MALib

MALib[2] integrates multiple benchmark environments and classical RL algorithms implementation, which we have omitted for simplicity. This means that MALib is a far more general framework while ours merely contains modules necessary for our experimental purpose. Our amendments can be summarized as follows:

1) L-MALib inherits the idea of centralized data server and policy server and enables much simpler data storage and adds limiters for various sampling schemes.

2) L-MALib optimizes and evaluates performance of asynchronous rollout and training (similar to IMPALA[32]) on multi-node and multi-GPU machines.

3) L-MALib adds many modifications to the GRF environment, such as statistics analysis, reward shaping, etc.

4) L-MALib makes policy architecture extensible for various settings, such as multi-headed, dimension extension, etc.

5) L-MALib adds a league training[7] framework.

A.3 Environment setting

We focus on 11-vs-11 multi-agent scenarios with 10 players each team controllable[4] (excluding the goalkeepers) as illustrated in Fig. A1. We mainly test on official 11-vs-11_hard_stochastic and 11-vs-11_kaggle scenarios. The observation is fully-observed for each agent, containing information on both teammates and opponents. The duration is 3 000 corresponding to 90 minutes in the real world. Each teams on both sides obtains more scores than the other side and will win this game. Otherwise, both teams make a tie when the scores are even. In addition, we insert done flags at the moment when one team scores to segment the long-horizon trajectory.



Fig. A1 Initial lineup of the 11-vs-11 scenarios where each team owns 11 players with 10 players controllable (except for the goalkeeper). In the figure, the left team is at the kickoff.

A.4 Experimental setup

In the following subsections, we list the default configurations in our experiments.

Feature encoding. The raw observation provided by Google Football is fully observable, containing information on player status from both teams, ball status, match state, etc. The official vectorized observation simple115_v2 with dimension 115 provided by GRF is hard to learn (experiments shown in Section 5.1) as it only contains the absolute position and direction of players and the ball. In real life, a football player might also be interested in its close teammates and opponents. Thus, we additionally add features of relative position in this regard. In particular, we borrowed the basic version of features from football-paris[39] which enlarged the dimension up to 217. For each single player, this will include the relative position of the current player and the status of the closest teammates and the opponents as shown in Table A2. We name it encoder_basic. We also construct an enhanced version named encoder_enhanced which includes extra information regarding off-side and match states.

Action masking. Action masking is essential for speeding up convergence. We use action masking similar to football-paris[39] which throws away possible undesired moves depending on the current state. It can be summarized as follows:

1) When one of our players owns the ball, we disable other players′ pass, shoot and dribble when the ball is far from them;

2) When the opponents own the ball, we disable a player′s slide when the ball is far from it;

3) When nobody owns the ball, we disable a player′s pass, shoot, dribble, and slide when the ball is far from it;

4) We disable a player′s shoot when the ball is away from the opponent′s penalty area and disable its long and high pass when within the penalty area;

5) We also apply action masks related to different game modes, e.g., the corner mode or the penalty mode.

We use this action masking as it can accelerate training agents to beat built-in AI with hard difficulty 1.0. However, when facing more skilled opponents, agents without action masking are more likely to learn finer control and strategies. Therefore, we remove masking in population-based training.

Reward shaping. The football game is a sparse reward problem, where agents can only receive scoring rewards when goals. Based on prior experience with football, we designed several reward functions and traced each reward factor back to the time step when the corresponding event occured. For instance, the player receives an assisting reward the moment he starts passing the ball to his teammate who scores afterward. At default, we use simple zero-sum goal/lose-goal team rewards for experiments against built-in AI as we found it sufficient, and complex rewards settings for population-based training including individual rewards for assisting and losing/gain-

[4] This setting aligns with Tikick[27]. The goalkeeper controlled by built-in logics can save the ball, but when controlled by the agent, its action space is aligned with other players and therefore cannot perform ball-saving anymore.

Table A2 Feature encoder implemented

Feature encoder	Encoded infos	Dimension
Encoder_basic	**Player_state:** left team position, direction, speed, roles, others **Ball state**: ball position, zone, relative position, direction, speed, ownership **Left team state**: teammate relative position, direction, speed, distance **Right team state**: opponent relative position, direction, speed, distance **Left closest state**: state of the closest teammate **Right closest state**: state of the closest opponent **Available action**: action mask	217
Encoder_enhanced	**Encoder_basic + offside infos + match state**	312
Simple 115v2	Position and direction of left and right team + ball_state + ownership encoding + active player encoding + game mode encoding + available action	115 + 19

ing balls to improve the skills of the policies.

Simple reward shaping is summarized in Table 3.

The complex reward shaping includes rewards for active offense, active defense and others. The active offense is designed to encourage attacks by:

1) Goal difference: The difference in goals between the left and right teams;

2) Win reward: The difference in goals but only given when winning the game;

3) Ball position: Obtain a positive reward when the ball is at the opponent′s half and the value increases if the ball is within the opponent′s penalty area, which encourages the player to move the ball forwards;

4) Goal pass: The number of passes before a goal, reset if the opponent scores. This encourages passing that helps to score;

5) Shot reward: Rewarding shot when it leads to a goal.

To encourage active defense, we have designed the following:

1) Minimum distance: The negative distance between the player and the closest opponent, which results in high-pressure defense.

2) Lost possession: Penalize if losing ball control.

3) Get possession: Reward if steel or intercept the ball.

4) Hold ball: Reward if we hold the ball and penalty if the opponent holds the ball.

Other reward shaping includes the following:

1) Role-based: Reward goal and penalize goal lose according to player′s role. The forwards obtains more goal rewards and less goal loss penalty, and the guard is the opposite. This helps players to play their corresponding roles.

2) Passing: Reward good passing and penalize bad passing.

Hyperparameter settings: The hyperparameter setting contains configurations for rollout, training, data collecting and algorithm setup as shown in Table A3.

Table A3 Hyperparameter setting

Rollout	Rollout length	3 000
	Sample length	1 000
	Env	11-vs-11 hard
	Action set	Default
Training	Local queue size	1
	Update interval	1
	Optimizer	Adam
	Actor_lr	5×10^{-4}
	Critic_lr	5×10^{-4}
	Optimizer eps	1×10^{-5}
Data server	Capacity	1 000
	Sampler type	Lumrf
Algorithm	Initialization	Orthogonal
	Gain	1
	Actor net	[256, 128, 64], ReLU
	Critic net	[256, 128, 64], ReLU
	PPO epoch	5
	Mini batch	1
	GAE lambda	0.95
	Feature norm	True
	PopArt beta	0.999 99
	Entropy coefficient	0.000 1
	Clip value	0.2
	KL early-stopping	0.01

References

[1] K. Kurach, A. Raichuk, P. Stańczyk, M. Zając, O. Bachem, L. Espeholt, C. Riquelme, D. Vincent, M. Michalski,

O. Bousquet, S. Gelly. Google research football: A novel reinforcement learning environment. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*, New York, USA, pp. 4501–4510, 2020. DOI: 10.1609/aaai.v34i04.5878.

[2] C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. S. Torr, M. F. Sun, S. Whiteson. Is independent learning all you need in the StarCraft multi-agent challenge? [Online], Available: https://arxiv.org/abs/2011.09533, 2020.

[3] M. Zhou, Z. Y. Wan, H. J. Wang, M. N. Wen, R. Z. Wu, Y. Wen, Y. D. Yang, W. N. Zhang, J. Wang. MALib: A parallel framework for population-based multi-agent reinforcement learning, [Online], Available: https://arxiv.org/abs/2106.07551, 2021.

[4] M. Jaderberg, W. M. Czarnecki, I. Dunning, L. Marris, G. Lever, A. G. Castaneda, C. Beattie, N. C. Rabinowitz, A. S. Morcos, A. Ruderman, N. Sonnerat, T. Green, L. Deason, J. Z. Leibo, D. Silver, D. Hassabis, K. Kavukcuoglu, T. Graepel. Human-level performance in first-person multiplayer games with population-based deep reinforcement learning, [Online], Available: https://arxiv.org/abs/1807.01281, 2018.

[5] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, D. Silver. Mastering Atari, Go, chess and Shogi by planning with a learned model. *Nature*, vol. 588, no. 7839, pp. 604–609, 2020. DOI: 10.1038/S41586-020-03051-4.

[6] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, vol. 529, no. 7587, pp. 484–489, 2016. DOI: 10.1038/nature16961.

[7] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Y. Wang, T. Pfaff, Y. H. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, vol. 575, no. 7782, pp. 350–354, 2019. DOI: 10.1038/s41586-019-1724-z.

[8] J. Kober, J. A. Bagnell, J. Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013. DOI: 10.1177/0278364913495721.

[9] H. Cai, K. Ren, W. N. Zhang, K. Malialis, J. Wang, Y. Yu, D. F. Guo. Real-time bidding by reinforcement learning in display advertising. In *Proceedings of the 10th ACM International Conference on Web Search and Data Mining*, ACM, Cambridge, UK, pp. 661–670, 2017. DOI: 10.1145/3018661.3018702.

[10] X. L. Huang, X. M. Ma, F. Hu. Editorial: Machine learning and intelligent communications. *Mobile Networks and Applications*, vol. 23, no. 1, pp. 68–70, 2018. DOI: 10.1007/s11036-017-0962-2.

[11] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*, Cambridge, USA: MIT Press, 1998.

[12] C. J. C. H. Watkins, P. Dayan. *Q*-learning. *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992. DOI: 10.1007/BF00992698.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms, [Online], Available: https://arxiv.org/abs/1707.06347, 2017.

[14] F. Christianos, G. Papoudakis, A. Rahman, S. V. Albrecht. Scaling multi-agent reinforcement learning with selective parameter sharing. In *Proceedings of the 38th International Conference on Machine Learning*, pp. 1989–1998, 2021.

[15] Google Research. Google research football with Manchester City F.C.: A word from Manchester City F.C., [Online], Available: https://www.kaggle.com/c/google-football, 2020.

[16] Google Research. Google research football with Manchester City F.C.: WeKick: Temporary 1st place Solution, [Online], Available: https://www.kaggle.com/c/google-football/discussion/202232, 2020.

[17] Institute of Automation, Chinese Academy of Sciences. IEEE CoG 2022 football AI competition, [Online], Available: http://www.jidiai.cn/compete_detail?compete=15, 2022. (in Chinese)

[18] C. H. Li, T. H. Wang, C. J. Wu, Q. C. Zhao, J. Yang, C. J. Zhang. Celebrating diversity in shared multi-agent reinforcement learning. In *Proceedings of the 35th Neural Information Processing Systems*, pp. 3991–4002, 2021.

[19] Y. R. Niu, R. Paleja, M. Gombolay. Multi-agent graph-attention communication and teaming. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, International Foundation for Autonomous Agents and Multiagent Systems, UK, pp. 964–973, 2021.

[20] J. Roy, P. Barde, F. G. Harvey, D. Nowrouzezahrai, C. Pal. Promoting coordination through policy regularization in multi-agent deep reinforcement learning. In *Proceedings of the 34th Neural Information Processing Systems*, Vancouver, Canada, Article number. 1323, 2020.

[21] D. Yang, Y. H. Tang. Adaptive inner-reward shaping in sparse reward games. In *Proceedings of International Joint Conference on Neural Networks*, IEEE, Glasgow, UK, 2020.

[22] J. Q. Ruan, Y. L. Du, X. T. Xiong, D. P. Xing, X. Y. Li, L. H. Meng, H. F. Zhang, J. Wang, B. Xu. GCS: Graph-based coordination strategy for multi-agent reinforcement learning. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems*, International Foundation for Autonomous Agents and Multiagent Systems, Auckland, New Zealand, pp. 1128–1136, 2022.

[23] L. Wang, Y. P. Zhang, Y. J. Hu, W. X. Wang, C. J. Zhang, Y. Gao, J. Y. Hao, T. J. Lv, C. J. Fan. Individual reward assisted multi-agent reinforcement learning. In *Proceedings of the International Conference on Machine Learning*, Baltimore, USA, pp. 23417–23432, 2022.

[24] H. Jiang, Y. T. Liu, S. Z. Li, J. Y. Zhang, X. H. Xu, D. H. Liu. Diverse effective relationship exploration for cooper-

ative multi-agent reinforcement learning. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, ACM, Atlanta, USA, pp. 842–851, 2022. DOI: 10.1145/3511808.3557292.

[25] Z. Q. Pu, H. M. Wang, B. Y. Liu, J. Q. Yi. Cognition-driven multi-agent policy learning framework for promoting cooperation. *IEEE Transactions on Games*, [Online], Available: https://ieeexplore.ieee.org/document/9807394.

[26] Y. Z. Niu, J. L. Liu, Y. H. Shi, J. R. Zhu. Graph neural network based agent in Google research football, [Online], Available: https://arxiv.org/abs/2204.11142, 2022.

[27] S. Y. Huang, W. Z. Chen, L. F. Zhang, S. Z. Xu, Z. Y. Li, F. M. Zhu, D. H. Ye, T. Chen, J. Zhu. TiKick: Towards playing multi-agent football full games from single-agent demonstrations, [Online], Available: https://arxiv.org/abs/2110.04507, 2021.

[28] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the 4th International Conference on Learning Representations*, San Juan, Puerto Rico, 2016.

[29] D. P. Kingma, J. Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, San Diego, USA, 2015.

[30] L. C. Dinh, Y. D. Yang, S. McAleer, Z. Tian, N. P. Nieves, O. Slumbers, D. H. Mguni, H. B. Ammar, J. Wang. Online double oracle, [Online], Available: https://arxiv.org/abs/2103.07780, 2021.

[31] M. Zhou, J. X. Chen, Y. Wen, W. N. Zhang, Y. D. Yang, Y. Yu, J. Wang. Efficient policy space response oracles, [Online], Available: https://arxiv.org/abs/2202.00633, 2022.

[32] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, K. Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning*, Stockholmsmässan, Sweden, pp. 1406–1415, 2018.

[33] S. McAleer, J. Lanier, R. Fox, P. Baldi. Pipeline PSRO: A scalable approach for finding approximate nash equilibria in large games. In *Proceedings of the 34th Neural Information Processing Systems*, Vancouver, Canada, Article number 1699, 2020.

[34] J. Scheiermann, W. Konen. AlphaZero-inspired general board game learning and playing, [Online], Available: https://arxiv.org/abs/2204.13307v1, 2022.

[35] R. Sanjaya, J. Wang, Y. D. Yang. Measuring the non-transitivity in chess. *Algorithms*, vol. 15, no. 5, Article number 152, 2022. DOI: 10.3390/a15050152.

[36] L. Han, J. C. Xiong, P. Sun, X. H. Sun, M. Fang, Q. W. Guo, Q. B. Chen, T. F. Shi, H. S. Yu, X. P. Wu, Z. Y. Zhang. TStarBot-X: An open-sourced and comprehensive study for efficient league training in StarCraft II full game, [Online], Available: https://arxiv.org/abs/2011.13729, 2020.

[37] D. Memmert, D. Raabe, S. Schwab, R. Rein. A tactical comparison of the 4-2-3-1 and 3-5-2 formation in soccer: A theory-oriented, experimental approach based on positional data in an 11 vs. 11 game set-up. *PLoS One*, vol. 14, no. 1, Article number e0210191, 2019. DOI: 10.1371/journal.pone.0210191.

[38] B. Low, D. Coutinho, B. Gonçalves, R. Rein, D. Memmert, J. Sampaio. A systematic review of collective tactical behaviours in football using positional data. *Sports Medicine*, vol. 50, no. 2, pp. 343–385, 2020. DOI: 10.1007/s40279-019-01194-7.

[39] seungeunrho. Kaggle football competition 6th solution, [Online], Available: https://github.com/seungeunrho/football-paris, 2020.

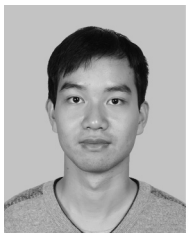


Yan Song received the B. Sc. degree in mathematics from the University of Manchester, UK in 2019 and the M. Sc. degree in Computational, Statistics and Machine Learning from the University College London, UK in 2020. He is currently a machine learning engineer at Institute of Automation, Chinese Academy of Sciences (CASIA), China.

His research interests include reinforcement learning and probabilistic inference.

Email: yan.song@ia.ac.cn

ORCID iD: 0000-0003-4216-7880



He Jiang received the B. Sc. degree in computer science and engineering from Shanghai Jiao Tong University in 2017 and the M. Sc. degree in control science and engineering from Tsinghua University in 2020. He is currently a research intern at Digital Brain Laboratory, China.

His research interests include multi-agent reinforcement learning and robotics.

Email: srevir@foxmail.com

ORCID iD: 0000-0001-7345-2225



Zheng Tian received the Ph. D. degree from University College London, UK in 2021. He is currently a machine learning researcher and an assistant professor at ShanghaiTech University, China. He studies reinforcement learning, multi-agent system and relevant applications in art and design. He has published several papers at top conferences, along with the best system paper award at CoRL 2020.

His research interests include reinforcement learning and multi-agent system and relevant applications in art and design.

Email: tianzheng@shanghaitech.edu.cn (Corresponding author)



Haifeng Zhang is an associate professor at Institute of Automation, Chinese Academy of Sciences (CASIA), China. He has published research papers on international conferences ICML, NeurIPS, AAAI, IJCAI, AAMAS etc. He has served as a reviewer for AAAI, IJCAI, TNNLS, Acta Automatica Sinica etc., and co-chair for IJCAI competition, IJTCS, DAI Workshop etc.

His research interests include reinforcement learning, game AI, game theory and computational advertising.

Email: haifeng.zhang@ia.ac.cn



Yingping Zhang received the B. Sc. and M. Sc. degrees in mechanical engineering from Central South University, China in 2016 and 2019, respectively. He is currently a machine learning engineer at Huawei Cloud, China.

His research interests include imperfect information game and multi agent reinforcement learning.

Email: zhangyingping@huawei.com



Jiangcheng Zhu received the B. Sc. and Ph. D. degree in automation from Zhejiang University, China in 2012 and 2019. He received the M. Sc. degree in the University of California at San Diego, USA in 2014. He is currently a senior AI scientist in Huawei Cloud, China.

His research interests include reinforcement learning and large-scale AI system.

Email: zhujiangcheng@huawei.com



Zonghong Dai received the B. Sc. degree in Nanjing Normal University, China in 2006. He is currently the CTO of AI at Huawei Cloud, China.

His research interests include large-scale AI system, AI domain generalization and decision intelligence.

Email: daizonghong@huawei.com



Weinan Zhang is now a tenure-track associate professor at Shanghai Jiao Tong University, China. He has published over 150 research papers on international conferences and journals and has been serving as an area chair or (senior) PC member at ICML, NeurIPS, ICLR, KDD, AAAI, IJCAI, SIGIR etc. and a reviewer at JMLR, TOIS, TKDE, TIST etc.

His research interests include reinforcement learning, deep learning and data science with various real-world applications.

Email: wnzhang@sjtu.edu.cn (Corresponding author)
ORCID iD: 0000-0002-0127-2425



Jun Wang is Chair Professor in Computer Science in University College London, and Founding Director of MSc Web Science and Big Data Analytics, UK. His team won the first global real-time bidding algorithm contest with 80 + participants worldwide. He has published over 100 research papers and is a winner of multiple "Best Paper" awards. He has served as an Area Chair in ACM CIKM and ACM SIGIR. His recent service includes co-chair of Artificial Intelligence, Semantics, and Dialog in ACM SIGIR 2018.

His main research interests are in the areas of AI and intelligent systems, including (multiagent) reinforcement learning, deep generative models, and their diverse applications on information retrieval, recommender systems and personalization, data mining, smart cities, bot planning, computational advertising, etc.

Email: jun.wang@cs.ucl.ac.uk